

9 AM - 10 AM	ASP.NET Core Overview	ASP.NET Core Overview Key Features of ASP.NET Core: a. Cross-Platform b. High Performance c. Modular and Lightweight d. Dependency Injection e. Open Source
--------------	-----------------------	--

ASP.NET Core Overview

ASP.NET Core is a modern, open-source, and cross-platform framework for building high-performance web applications, APIs, and microservices. It is developed by Microsoft and is a major redesign of the older **ASP.NET** framework. ASP.NET Core is part of the .NET platform, and it is built to be lightweight, modular, and capable of running on different operating systems such as Windows, Linux, and macOS.

Key Features of ASP.NET Core:

1. **Cross-Platform:**

ASP.NET Core applications can run on Windows, Linux, and macOS, making it easier to develop and deploy across different environments.

2. **High Performance:**

ASP.NET Core is optimized for high performance and scalability. The framework uses Kestrel, a high-performance web server, and is designed to handle a large number of concurrent requests efficiently.

3. **Modular and Lightweight:**

Unlike the traditional ASP.NET, which included many preinstalled components, ASP.NET Core is modular. You can install only the packages and features you need, reducing the overall application size and improving startup performance.

4. **Unified Framework:**

ASP.NET Core combines MVC (Model-View-Controller) and Web API functionality into a single framework, making it easier to develop both web applications and RESTful APIs using the same tools.

5. **Dependency Injection (DI):**

ASP.NET Core has built-in support for dependency injection, which helps in writing more testable and maintainable code by allowing classes to receive dependencies via constructor injection.

6. **Open Source:**

ASP.NET Core is open-source and hosted on GitHub, which allows developers to contribute to its development and use the framework for free.

7. Cloud-Ready:

ASP.NET Core is designed to work well with cloud platforms like **Azure** and supports containerization via **Docker**, making it suitable for cloud-based applications and microservices.

8. Asynchronous Programming:

ASP.NET Core supports asynchronous programming out-of-the-box, allowing you to handle I/O-bound tasks (such as database queries, file reads, or HTTP requests) more efficiently without blocking threads.

9. Security:

ASP.NET Core comes with a variety of built-in security features, such as authentication (OAuth, OpenID Connect), authorization, data protection, HTTPS enforcement, and Cross-Site Request Forgery (CSRF) protection.

10. Built-in Support for Modern Web Development:

ASP.NET Core provides excellent support for modern web standards such as HTML5, CSS3, and JavaScript. It also supports front-end frameworks like **React**, **Angular**, and **Vue.js** by providing tools like **Razor Pages** and **Blazor** for building interactive web applications.

Components of ASP.NET Core

10 AM - 11 AM	ASP.NET Core Overview	Components of ASP.NET Core a. ASP.NET Core MVC b. ASP.NET Core Razor Pages c. ASP.NET Core Web API: d. Entity Framework Core (EF Core) e. Dependency Injection f. Middleware
---------------	-----------------------	--

1. ASP.NET Core MVC:

- ASP.NET Core MVC is used for building web applications following the **Model-View-Controller (MVC)** architectural pattern.
- It is ideal for traditional web applications with views rendered on the server.
- The controller handles user input, the model represents the data, and the view is the template rendered as HTML.

Example:

csharp

Copy code

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

2. ASP.NET Core Razor Pages:

- Razor Pages is an alternative to MVC that is more page-centric. It is suitable for applications where each page is an independent unit.
- Razor Pages are modeled after the MVC structure but without the need to explicitly separate controllers.

Example:

csharp

Copy code

@page

@model IndexModel

<h1>@Model.Message</h1>

3. ASP.NET Core Web API:

- ASP.NET Core allows you to build RESTful APIs using the same underlying framework, which simplifies the development of APIs alongside traditional MVC applications.
- APIs are created by defining **Controllers** that return **JSON** or **XML** responses to clients.

Example:

csharp

Copy code

[ApiController]

```
[Route("api/[controller]")]
```

```
public class WeatherForecastController : ControllerBase
```

```
{
```

```
    private static readonly string[] Summaries = new[]
```

```
    {
```

```
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering",  
        "Scorching"
```

```
    };
```

```
    private readonly ILogger<WeatherForecastController> _logger;
```

```
    public WeatherForecastController(ILogger<WeatherForecastController> logger)
```

```
    {
```

```
        _logger = logger;
```

```
    }
```

```
    [HttpGet]
```

```
    public IEnumerable<WeatherForecast> Get()
```

```
    {
```

```
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
```

```
        {
```

```
            Date = DateTime.Now.AddDays(index),
```

```
            TemperatureC = Random.Shared.Next(-20, 55),
```

```
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
```

```
        })
```

```
        .ToArray();
```

```
    }
```

```
}
```

11 AM - 12 PM	ASP.NET Core Overview	Components of ASP.NET Core d. Entity Framework Core (EF Core) e. Dependency Injection f. Middleware
---------------	-----------------------	--

4. Entity Framework Core (EF Core):

- EF Core is an object-relational mapping (ORM) framework for working with databases. It simplifies database interactions by allowing developers to work with data as objects, eliminating the need for raw SQL in many cases.
- EF Core supports multiple database providers, such as **SQL Server**, **SQLite**, **PostgreSQL**, and **MySQL**.

5. Dependency Injection:

- ASP.NET Core has built-in **Dependency Injection (DI)**, which simplifies the management of dependencies (services, data access objects, etc.) throughout the application.
- You can register services in the **Startup.cs** class and inject them into controllers or other classes as needed.

6. Middleware:

- In ASP.NET Core, middleware components are used to handle requests and responses. They are configured in **the Configure method in Startup.cs**.
- Middleware is invoked in a pipeline, where each component performs specific tasks, such as authentication, logging, or error handling.

12PM - 1 PM	Practice Session
1PM - 2 PM	

2 PM - 3 PM	ASP.NET Core Environment Variables & Use of Environment Variables in ASP.NET Core	ASPNETCORE_ENVIRONMENT a. Development b. Staging c. Production ASPNETCORE_URLS DATABASE_URL DOTNET_RUN_CONFIGURATION: a. Debug b. Release
-------------	---	---

Common ASP.NET Core Environment Variables

1. ASPNETCORE_ENVIRONMENT:

- The ASPNETCORE_ENVIRONMENT variable is the most common environment variable used to specify the environment in which the application is running.
- It is used to determine the application's configuration and behavior (for example, to load different settings for development, staging, and production).

Common values for ASPNETCORE_ENVIRONMENT:

- Development
- Staging
- Production

Example:

Set the environment variable:

- **Windows (Command Prompt):**

```
bash
```

Copy code

```
set ASPNETCORE_ENVIRONMENT=Development
```

- **Linux/macOS:**

```
bash
```

Copy code

```
export ASPNETCORE_ENVIRONMENT=Development
```

2. ASPNETCORE_URLS:

- The ASPNETCORE_URLS environment variable is used to define the URLs that the ASP.NET Core application should listen to. You can set this to specify different ports and IP addresses.

Example:

- **Windows:**

bash

Copy code

```
set ASPNETCORE_URLS="http://localhost:5000"
```

- **Linux/macOS:**

bash

Copy code

```
export ASPNETCORE_URLS="http://localhost:5000"
```

3. DATABASE_URL (Example):

- While not built-in, developers often use environment variables to store configuration information, such as database connection strings.

Example:

- **Windows:**

bash

Copy code

```
set DATABASE_URL="Server=myServerAddress;Database=myDataBase;User  
Id=myUsername;Password=myPassword;"
```

- **Linux/macOS:**

bash

Copy code

```
export DATABASE_URL="Server=myServerAddress;Database=myDataBase;User  
Id=myUsername;Password=myPassword;"
```

4. DOTNET_RUN_CONFIGURATION:

- This environment variable can be used to specify the configuration (e.g., Release or Debug) when running the application.

Example:

- **Windows:**

bash

Copy code

```
set DOTNET_RUN_CONFIGURATION=Release
```

- **Linux/macOS:**

bash

Copy code

```
export DOTNET_RUN_CONFIGURATION=Release
```

5. Logging Configuration:

- ASP.NET Core applications can use environment variables to configure logging levels.
- For example, setting the Logging__LogLevel__Default environment variable can specify the log level for the application.

Example:

- **Windows:**

bash

Copy code

```
set Logging__LogLevel__Default=Information
```

- **Linux/macOS:**

bash

Copy code

```
export Logging__LogLevel__Default=Information
```

How to Use Environment Variables in ASP.NET Core

1. Configuring the Application Environment

ASP.NET Core uses the `IWebHostEnvironment` interface to get information about the environment in which the application is running. The value of the `ASPNETCORE_ENVIRONMENT` variable is used to determine the environment.

- **Accessing Environment Information:** You can access the current environment from the `IWebHostEnvironment` instance in your application.

Example:

csharp

Copy code

```
public class Startup
{
    private readonly IWebHostEnvironment _env;

    public Startup(IConfiguration configuration, IWebHostEnvironment env)
    {
        Configuration = configuration;
        _env = env; // Access the environment
    }

    public IConfiguration Configuration { get; }

    public void ConfigureServices(IServiceCollection services)
    {
        if (_env.IsDevelopment())
        {
            // Add services specific to Development
            services.AddScoped<IEmailService, DevEmailService>();
        }
        else if (_env.IsProduction())
        {
            // Add services specific to Production
            services.AddScoped<IEmailService, ProdEmailService>();
        }
    }
}
```

```
// Add services specific to Production
services.AddScoped<IEmailService, ProdEmailService>();
}
}
}
```

2. Configuration Based on Environment

ASP.NET Core uses environment variables along with `appsettings.json` and `appsettings.{Environment}.json` files to load configuration. For example, in the Development environment, the application will load settings from `appsettings.Development.json`.

- Example of **appsettings.json**:

json

Copy code

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

- Example of **appsettings.Development.json** (to be used in development environment):

json

Copy code

```
{
  "Logging": {
```

```
"LogLevel": {  
  "Default": "Debug"  
}  
}  
}
```

The environment-specific settings can be added to **appsettings.Production.json** and **appsettings.Staging.json** for those respective environments.

3. Setting and Reading Environment Variables

- **Setting environment variables:**

- On Windows, you can set them in Command Prompt:

```
bash
```

Copy code

```
set ASPNETCORE_ENVIRONMENT=Development
```

- On macOS or Linux, you can set them in the terminal:

```
bash
```

Copy code

```
export ASPNETCORE_ENVIRONMENT=Development
```

- **Reading environment variables** in C#: Use IConfiguration to read environment variables in your code.

Example:

```
csharp
```

Copy code

```
public class MyController : Controller  
{  
  private readonly IConfiguration _configuration;  
  
  public MyController(IConfiguration configuration)  
  {  
    _configuration = configuration;  
  }  
}
```

```
}

public IActionResult Index()
{
    // Reading an environment variable

    string envValue = _configuration["MY_CUSTOM_ENV_VARIABLE"];

    return Content($"The value of MY_CUSTOM_ENV_VARIABLE is: {envValue}");
}
}
```

How Environment Variables Affect Application Behavior

- **Startup Configuration:** ASP.NET Core applications can conditionally load different services, configuration settings, or middleware based on the environment (e.g., Development, Staging, Production).
 - **Configuration Providers:** ASP.NET Core allows configuration to be read from multiple sources, including environment variables, configuration files, command-line arguments, and user secrets. The configuration system merges these sources together, allowing for easy overrides.
 - **Environment-Specific Settings:** By setting ASPNETCORE_ENVIRONMENT, you can ensure the application uses environment-specific settings such as different logging levels, feature flags, or external service configurations.
-

Best Practices for Using Environment Variables

1. **Sensitive Information:** Use environment variables for sensitive information like API keys, connection strings, and passwords. This avoids hardcoding sensitive data in code files.
2. **Environment-Specific Settings:** Use ASPNETCORE_ENVIRONMENT to load environment-specific configuration files (appsettings.{Environment}.json) and adjust application behavior accordingly.
3. **Deployment:** In cloud environments like **Azure**, environment variables can be set through the cloud provider's configuration system. For local development,

you can use tools like **Docker**, **Visual Studio**, or **dotnet user-secrets** for managing these variables.

Conclusion

Environment variables in **ASP.NET Core** offer flexibility, security, and ease of configuration management. They help in creating applications that behave differently in development, staging, and production environments without modifying the code itself. Using environment variables, developers can store sensitive data securely, manage environment-specific settings, and ensure that applications are scalable and adaptable across various environments.

3 PM - 4 PM	Create ASP.NET Core Web Application	Create Simple ASP.Net Core Web Application Install .NET SDK Create a New Project Run the Application View in Browser
-------------	-------------------------------------	--

Creating a Simple ASP.NET Core Web Application

1. Step 1: Install .NET SDK

Download and install the .NET SDK from the official .NET website:

dotnet.microsoft.com

2. Step 2: Create a New Project

Open a terminal or command prompt, and run the following command to create a new ASP.NET Core web application:

```
bash
```

Copy code

```
dotnet new mvc -n MyFirstApp
```

```
cd MyFirstApp
```

3. Step 3: Run the Application

After the project is created, run the following command to build and run the application:

bash

Copy code

dotnet run

4. **Step 4: View in Browser** Once the application is running, open a browser and go to `http://localhost:5000` to see the application in action.

Structure of an ASP.NET Core Project

An ASP.NET Core project typically contains the following folders and files:

- **Controllers/**: Contains controller classes that handle incoming requests and return responses.
- **Views/**: Contains Razor view pages (HTML + C#) for rendering HTML responses.
- **Models/**: Contains classes that represent the application's data and business logic.
- **wwwroot/**: Contains static files such as images, CSS, JavaScript, and other assets.
- **appsettings.json**: Configuration file for application settings such as connection strings and app-specific settings.
- **Program.cs**: Contains the entry point for the application.
- **Startup.cs**: Configures services and middleware for the application (depending on the version of .NET, in newer versions of .NET 6+, the configuration is often handled in Program.cs).

ASP.NET Core Environment Variables

In **ASP.NET Core**, environment variables play a key role in configuring and controlling the behaviour of an application based on the environment it is running in. This allows developers to create flexible and configurable applications that can behave differently in development, staging, and production environments without changing the code.

Environment variables are typically used for:

- Storing configuration settings (e.g., database connection strings, API keys).
- Switching between different configurations for different environments (e.g., **Development, Staging, Production**).
- Controlling application behaviour, such as enabling or disabling certain features.

ASP.NET Core provides built-in support for environment-based configuration and seamlessly integrates with environment variables to enable these use cases.

4 PM - 5 PM	Practice Session
5 PM - 6 PM	HR Trainings
6 PM - 7 PM	HR Trainings